



Cours 6:

Partie 2: langage de définition de données (DDL)

Enseignante responsable:
Dr. Nesrine LAHIANI

Public cible : 2^{ème} année Ingénieur

Contact : lahiani.nesrine@gmail.com

Langage de définition de données

Le langage de définition de données comprend les instructions suivantes:

- **CREATE**, pour créer de nouveaux objets du schéma (tables, types, index ...).
- **DROP** pour supprimer des objets existants.
- **ALTER** pour modifier des objets existants.

Instruction CREATE

- Permet de créer de nouveaux objets du schéma (tables, types ...).
- A ne pas confondre avec l'instruction ~~INSERT~~.

➤ **Créer** une Bases de données :
CREATE DATABASE nom_bdd ;

➤ **Supprimer** une Bases de données
DROP DATABASE nom_bdd;

➤ Pour créer une nouvelle table:

```
CREATE TABLE nom_table (  
    colonne1 type1,  
    colonne2 type2  
    ...  
)
```

Types de données

La norme SQL2 prévoit un grand nombre de types de données:

- Numériques
- Caractères
- Temporels
- Binaires

Les types numériques

Les types de données numériques sont divisés en 8 catégories :

- **Tinyint** : étendue de 0 à 255 ;
- **Smallint** : de -32,768 à 32,767 ;
- **Int** : de -2,147,483,648 à 2,147,483,647 ;
- **Bigint** : de -9,223,372,036,854,775,808 à 9,223,372,036,854,775,807 ;
- **Decimal** : de $-10^{38} + 1$ à $10^{38} - 1$;
- **Numeric** : de $-10^{38} + 1$ à $10^{38} - 1$;
- **Float** : de $-1.79E + 308$ à $1.79E + 308$;
- **Real** : de $-3.40E + 38$ à $3.40E + 38$.

NUMERIC(10, 2) : Cela signifie qu'il y a 10 chiffres au total, avec 2 chiffres après la virgule.
DECIMAL(8, 4) : Cela signifie qu'il y a 8 chiffres au total, avec 4 chiffres après la virgule.

Les types chaînes de caractères

- Sont entourées par des apostrophes. Si la chaîne contient une apostrophe, celle-ci devra être doublée.
- Il existe deux types :
 - pour les chaînes de longueur constante (inférieur à 2000 caractères, pour ORACLE) CHAR (longueur) :
 - pour les chaînes de longueur variable (Tous les SGBD ont une longueur maximale pour ces chaînes, 4000 sous ORACLE)
 - VARCHAR (longueur) ou
 - VARCHAR2(longueur) sous ORACLE

Longueur est le nombre maximale de caractères qu'il sera possible de stocker dans le champ

Les types temporels

SQL2

- **DATE** : réserve 2 chiffres pour le mois et le jour et 4 pour l'année
- **TIME** : pour les heures, minutes et secondes
- **TIMESTAMP** : indique un moment précis par une date avec heures, minutes et secondes (6 chiffres après la virgule)

Oracle

DATE : chaîne de caractères entre apostrophes. Le format dépend des options que l'administrateur a choisies au moment de la création de la base.

Un type DATE inclut un temps en heures, minutes et secondes.

Les types binaires

- **SQL2 n'a pas normalisé ce type de données**
- **Ce type permet d'enregistrer des données telles que les images et les sons de très grande taille avec divers formats**
- **Les différents SGBD fournissent un type pour ces données.**

Les noms varient :

- ✓ **LONG RAW pour Oracle**
- ✓ **IMAGE pour Sybase**
- ✓ **BYTE pour Informix**

Instruction CREATE

```
CREATE TABLE Projet(  
    Num_Projet INT,  
    Description CHAR (200),  
    Date_Début DATE,  
    Date_Fin DATE,  
    Budget NUMERIC (8,2)  
)
```

Instruction CREATE

Pour créer une nouvelle table avec contraintes d'intégrité sur attribut:

```
CREATE TABLE nom_table (  
    colonne1 type1 [contrainte d'attribut1],  
    colonne2 type2 [contrainte d'attribut2]  
    ...  
)
```

Contraintes d'intégrité

- **Contrainte de clé primaire** (PRIMARY KEY)
La clé primaire de chaque table doit être spécifiée explicitement
- **Contrainte de non vacuité** (NOT NULL)
Les attributs devant avoir une valeur connue
- **Contrainte d'unicité** (UNIQUE)
Les attributs devant être uniques doivent également être spécifiés
- **Contrainte de domaine** (CHECK)
La valeur d'un attribut doit être comprise dans un domaine de valeurs
- **Contrainte référentielle** (FOREIGN KEY)
La clé étrangère d'une relation doit obligatoirement référencer une valeur existante

La contrainte **PRIMARY KEY** regroupe les contraintes **UNIQUE** et **NOT NULL**.

Instruction CREATE

```
CREATE TABLE Projet(  
    Num_Projet INT PRIMARY KEY,  
    Description CHAR(200) NOT NULL,  
    Date_Début DATE NOT NULL,  
    Date_Fin DATE,  
    Budget NUMERIC(8, 2) DEFAULT 20000.00  
)
```

Si un nouveau tuple est inséré sans la valeur de l'attribut Budget, il prendra la valeur par défaut 20000.00

Instruction CREATE

```
CREATE TABLE Employé(  
    Num_Employé INT PRIMARY KEY,  
    Nom CHAR(30) NOT NULL,  
    Prénom CHAR(30),  
    Date_Naissance DATE,  
    Fonction CHAR(15) CHECK (Fonction IN ('Analyste', 'Chef de projet',  
                                           'Concepteur', 'Développeur')),  
    Est_Cadre CHAR(1) DEFAULT '0' CHECK (Est_Cadre IN ('0', '1'))  
)
```

Bien que présent sur certains SGBDR, le type booléen n'est pas spécifié par la norme SQL2

Instruction CREATE

```
CREATE TABLE Affectation(
```

```
    Num_Employé INT REFERENCES Employé(Num_Employé),  
    Num_Projet INT REFERENCES Projet(Num_Projet),  
    Début_Affect DATE NOT NULL,  
    Fin_Affect DATE,  
    Supérieur INT REFERENCES Employé(Num_Employé),  
    PRIMARY KEY(Num_Employé, Num_Projet)
```

```
)
```

Quand une contrainte doit s'appliquer à plusieurs attributs, elle doit être exprimée comme contrainte de table.

Instruction CREATE

```
CREATE TABLE nom_table (  
  colonne1 type1 [contrainte d'attribut1],  
  colonne2 type2 [contrainte d'attribut2] ...,  
  [contrainte1 de table]  
  [, contrainte2 de table]  
  ...  
)
```

Instruction CREATE

```
CREATE TABLE Affectation(  
    Num_Employé INT REFERENCES Employé(Num_Employé),  
    Num_Projet INT REFERENCES Projet(Num_Projet),  
    Début_Affect DATE NOT NULL,  
    Fin_Affect DATE,  
    Supérieur INT CONSTRAINT clé_étrangère_sup REFERENCES  
    Employé(Num_Employé),  
  
    CONSTRAINT clé_affectation PRIMARY KEY(Num_Employé,  
    Num_Projet)  
)
```

Toutes les contraintes sur attribut (ou sur table) peuvent être nommées.

Contraintes sur suppression de tuples

Il est en principe interdit de supprimer un tuple référencé par un autre tuple dans le schéma relationnel.

Avant de supprimer le tuple référencé, il faut d'abord supprimer tous les tuples qui le référencent.

Exemple:

Avant de supprimer l'employé 1023, il faut supprimer toutes ses affectations.

DELETE FROM Employé WHERE Num_Employé = 1023
Suppression impossible car le tuple employé est référencé par 2 tuples affectations

Num_Employé	Nom	Prénom	Date_Naissance	Fonction	Est_Cadre
1001	Belaid	Toufik	12/05/1965	Concepteur	1
1009	Touati	Rachid	13/09/1941	Chef de projet	1
1023	Kadri	Amine	23/11/1970	Développeur	1
1053	Djabi	Fatiha	04/06/1980	Analyste	0
1026	Bouras	Kamel	19/04/1968	Administrateur	1
1005	Djabi	Fatiha	22/08/1976	Développeur	0

Num_Employé	Num_Projet	Début_Affect	Fin_Affect	Supérieur
1009	122	07/03/2011	13/11/2011	NULL
1001	122	08/03/2011	28/06/2011	1009
1023	122	15/06/2011	04/10/2011	1009
1009	103	12/09/2010	01/11/2010	NULL
1001	208	15/06/2011	12/10/2011	1009
1009	208	15/06/2011	06/03/2012	NULL
1023	208	01/09/2011	17/12/2011	1009
1009	133	06/11/2011	19/02/2012	NULL
1053	208	01/09/2011	06/03/2012	1026
1026	208	19/08/2011	06/03/2012	1009

Suppression en cascade

La clause ON DELETE CASCADE permet de supprimer en cascade tous les tuples référençant un tuple supprimé.

```
CREATE TABLE Affectation(  
  Num_Employé INT REFERENCES Employé(Num_Employé) ON  
  DELETE CASCADE,  
  Num_Projet INT FOREIGN KEY REFERENCES Projet(Num_Projet),  
  Debut_Affect DATE NOT NULL,  
  Fin_Affect DATE,  
  Supérieur INT REFERENCES Employé(Num_Employé),  
  PRIMARY KEY (Num_Employé, Num_Projet))
```

DELETE FROM Employé WHERE Num_Employé = 1023

Suppression du tuple employé 1023 et de tous les tuples qui le référencent dans la table affectation.

Num_Employé	Nom	Prénom	Date_Naissance	Fonction	Est_Cadre
1001	Belaid	Toufik	12/05/1965	Concepteur	1
1009	Touati	Rachid	13/09/1941	Chef de projet	1
1023	Kadri	Amine	23/11/1970	Développeur	1
1053	Djabi	Fatiha	04/06/1980	Analyste	0
1026	Bouras	Kamel	19/04/1968	Administrateur	1
1005	Djabi	Fatiha	22/08/1976	Développeur	0

Num_Employé	Num_Projet	Début_Affect	Fin_Affect	Supérieur
1009	122	07/03/2011	13/11/2011	NULL
1001	122	08/03/2011	28/06/2011	1009
1023	122	15/06/2011	04/10/2011	1009
1009	103	12/09/2010	01/11/2010	NULL
1001	208	15/06/2011	12/10/2011	1009
1009	208	15/06/2011	06/03/2012	NULL
1023	208	01/09/2011	17/12/2011	1009
1009	133	06/11/2011	19/02/2012	NULL
1053	208	01/09/2011	06/03/2012	1026
1026	208	19/08/2011	06/03/2012	1009

Suppression et mise à jour

En plus de la clause ON DELETE CASCADE, la norme SQL2 prévoit les clauses suivantes:

- ON DELETE SET NULL
- ON DELETE SET DEFAULT
- ON UPDATE CASCADE
- ON UPDATE SET NULL
- ON UPDATE SET DEFAULT

Instruction ALTER

L'instruction ALTER permet de modifier le schéma d'une table.

Modifier le schéma d'une table consiste à effectuer un des changements suivants:

- Ajouter une ou plusieurs colonnes (**ADD** (col1 type1[, col2 type2 ...]))
- Modifier une ou plusieurs colonnes (**MODIFY** (col1 type 1[, col2 type2...]))

Instruction ALTER

- Supprimer une ou plusieurs colonnes (**DROP COLUMN** (col1 [,col2 ...]))
- Ajouter une contrainte (**ADD CONSTRAINT ...**)
- Supprimer une contrainte (**DROP CONSTRAINT** nom_contrainte)
- ...

Instruction ALTER

■ Exemples:

1) Ajouter une colonne Salaire à la table Employé

ALTER TABLE Employé ADD Salaire NUMBER(7,2)

2) Modifier le degré de précision de la colonne Budget et augmenter la taille maximale de la colonne Description de projet

**ALTER TABLE Projet MODIFY (Budget NUMBER(10,3),
Description VARCHAR2(255))**

3) Supprimer la colonne Date_Fin

ALTER TABLE Projet DROP COLUMN Date_Fin

Instruction ALTER

Ajouter une contrainte sur le Budget

```
ALTER TABLE Projet ADD CONSTRAINT Budget_Min CHECK(Budget >= 20000)
```

On ne peut ajouter une contrainte que si toutes les données déjà existantes vérifient cette contrainte.

Num_Projet	Description	Date_Début	Date_Fin	Budget
103	Mise en place d'un réseau intranet	12/09/2010	05/12/2010	39000.00
122	Développement d'une application de gestion	07/01/2011	13/05/2011	86000.00
133	Réalisation d'un CD-ROM interactif de formation	04/11/2011	02/07/2012	15000.00
208	Développement d'un site internet	15/06/2011	06/03/2012	50000.00

Instruction DROP

L'instruction **DROP** permet de **supprimer** un objet du schéma relationnel (table, vue...).

Supprime la table Employé et tous les tuples qu'elle contient.
A ne pas confondre avec l'instruction DELETE.

Pour supprimer une table:
`DROP TABLE nom_table`

Exemple:
`DROP TABLE Employé`

Instruction DROP

Il est interdit de supprimer une table référencée par une autre table.

Pour supprimer les tables Employé, Projet et Affectation, il faut d'abord supprimer la table Affectation et ensuite les tables Employé, Projet.

```
DROP TABLE Affectation;  
DROP TABLE Employé;  
DROP TABLE Projet;
```

Sinon, il faut d'abord supprimer les contraintes d'intégrité référençant une table avant de supprimer cette table.

Instruction DROP

L'option **CASCADE CONSTRAINTS** permet de supprimer les contraintes d'intégrité avant de supprimer les tables.

```
DROP TABLE Employé CASCADE  
CONSTRAINTS;
```

```
DROP TABLE Projet CASCADE CONSTRAINTS;  
DROP TABLE Affectation;
```